C-DAC Hyderabad - DBDA Project 2026

Multi-Agent AI Data Lakehouse Platform for Betting Site Data Intelligence

PRESENTATION ORAL SCRIPT & TECHNICAL DEFENSE WORKBOOK

(Slides 1-9 & 16-24 Walkthrough)

Guided By: Ms. Krishnaveni

Presented By Team BDA:

Ms. Riya Modi (260250325032)

Mr. Sri Charan (260250325017)

Mr. Prasad P. Gautre (260250325029)

Mr. Niraj S. Kadam (260250325025)

SLIDE 1: COVER PAGE / TITLE SLIDE

What It Is: The introductory slide of the presentation containing project title, guide name, BDA course details, and team presenter names.

Why This Project?: Digital payment tracking is critical for financial intelligence. Betting platforms constantly rotate bank accounts and UPI IDs to bypass regulatory bans. This project builds an automated platform to track and analyze these rotated accounts in real-time.

Spoken Script:

> *"Good morning/afternoon, respected guide Ms. Krishnaveni and members of the evaluation panel. I a*
> *and along with my team members-Sri Charan, Prasad Gautre, and Niraj Kadam-we are presenting*
> *project: 'Multi-Agent AI Data Lakehouse Platform for Betting Site Data Intelligence'.*
>
> *We chose this project to automate the detection and tracking of rotated merchant payment me*
> *dynamic betting websites in real-time."*

SLIDE 2: PROBLEM STATEMENT

What It Is: The core challenges the project addresses, covering dynamic layouts, endpoint rotation, and DOM noise.

How It Works: Online platforms hide payment options in dynamic scripts/iframes and rotate them frequently to avoid bans. Manual verification is slow and does not scale.

Why This: Standard database queries or single-site scrapers fail. We built a coordinated, multi-agent pipeline to decouple web collection from storage and risk modeling.

Why Not Other: Static SQL databases cannot handle dynamic website layout changes without schema crashes. Medallion architecture partitions raw data from refined tables.

Spoken Script:

> *"Betting platforms hide payment gateways inside dynamic web layouts and nested iframes, and fre*
> *their UPI IDs and wallets to avoid bans. This results in noisy, duplicate records.*
>
> *Checking these sites manually is slow, error-prone, and cannot scale. As shown in the workflow di*
> *our platform automates the entire process: scraping dynamic pages, streaming with Kafka,*
> *Lakehouse, and risk intelligence using ML and RAG."*

SLIDE 3: PROJECT OBJECTIVES

What It Is: The defined goals and roadmap of the Multi-Agent AI Data Lakehouse Platform.

How It Works: A step-by-step pipeline from raw web ingestion to clean storage, risk modeling, and a local RAG assistant.

Why This: Follows the industry-standard Medallion pipeline (Ingest -> Stream -> Clean -> Model -> Search -> Verify -> Visualize) ensuring data quality at each layer.

Spoken Script:

> *"Our primary goals are: first, automate dynamic data extraction; second, stream records via Kafka w loss; third, organize data into raw Bronze and clean Silver Parquet formats via MinIO; fourth, ap Learning to predict categories and detect anomalies; and finally, host a local Llama 3 RAG QA p Streamlit dashboard."*

SLIDE 4: SYSTEM OVERVIEW WORKFLOW

What It Is: The end-to-end processing pipeline architecture showing data transformations.

How It Works: Web Scraper -> Kafka Topic -> MinIO Bronze -> PySpark Silver -> Scikit-Learn ML -> FAISS Index -> LangChain local RAG -> Two-stage Verification -> Streamlit UI.

Why This: Decoupling components. If a web scraper encounters a layout change, the Kafka queue caches the message streams, keeping the downstream database, ML, and dashboard running.

Spoken Script:

> *"This slide represents our system overview workflow. Raw scraped data is sent as JSON messages t*
> *to land in our MinIO Bronze S3 store. PySpark cleans this data and writes it to Silver Parquet. Scik*
> *our risk models, while the records are embedded into a FAISS vector database. LangChain then*
> *context to feed our local LLM."*

SLIDE 5: TECHNOLOGY STACK MATRIX

What It Is: A matrix of the open-source libraries and enterprise components used in the pipeline.

Why This: MinIO provides local S3-compatible storage. PySpark provides in-memory computing for cleaning. Ollama/Llama 3 runs on-premises to ensure complete privacy.

Why Not Other: AWS/Databricks charge heavy fees; MinIO/Spark are free. OpenAI APIs require sending bank details to external servers, violating GDPR/PCI-DSS rules. Local LLMs are completely secure.

Spoken Script:

> *"Here is our technology matrix. We chose a 100% open-source stack that runs locally on standard using local Llama 3 via Ollama, we keep all merchant bank accounts and UPI IDs private on-premi cloud API costs."*

SLIDE 6: THE 6-AGENT PLATFORM ARCHITECTURE

What It Is: The multi-agent design dividing responsibilities across 6 specialized software agents.

How It Works: Coordination: Agent 1 Scrapes -> Agent 2 Streams -> Agent 3 Spark Cleans -> Agent 4 ML Runs -> Agent 5 RAG Agent (FAISS + LLM) -> Agent 6 UI Dashboard. Note: Verification Agent runs silently in the backend.

Why This: Modularity. If a website selector breaks, we only edit Agent 1's rules without touching the ML classification in Agent 4 or the RAG in Agent 5.

Why Not Other: Traditional monolithic architectures merge all code into one giant script, making debugging difficult and increasing system failure risk.

Spoken Script:

> *"Our platform divides responsibilities across six specialized agents: Agent 1 scrapes data, Agent Agent 3 cleans it, Agent 4 runs machine learning, Agent 5 handles semantic vector RAG with Llama 6 serves as our master UI orchestrator. Our verification check runs implicitly in the background to security."*

SLIDE 7: COMPLETE WORKFLOW IN BRIEF

What It Is: A concise summary of the linear database state transitions.

How It Works: Live Site -> JSON Event -> Kafka Ingestion -> MinIO Bronze JSON -> PySpark Silver Parquet -> Machine Learning Analytics -> FAISS Indexing -> LangChain Retrieval -> Verified UI.

Spoken Script:

> *"This slide summarizes our workflow in brief. The data moves linearly: live site details are scrap*
> *events, streamed via Kafka into MinIO Bronze, cleaned by PySpark into Silver Parquet, analyzed by*
> *embedded into FAISS, retrieved by LangChain, and verified for the analyst UI."*

SLIDE 8: AGENT 1: DATA COLLECTION SCRAPER

What It Is: The automated data collection agent powered by Playwright and Selenium.

How It Works: Headless Chrome controls navigate betting modals, switch iframes, and pyzbar extracts UPI VPAs from canvas QR code screenshots.

Why This: Standard tools (like BeautifulSoup) cannot render JS or switch inside dynamic cross-origin iframes where payment elements reside.

Why Not Other: Requests/BeautifulSoup only load static HTML. Paid scrapers (like Browserless) cost money. Playwright and Selenium are 100% free and easily containerized.

Spoken Script:

> *"Let's look at Agent 1, our Data Collection Scraper. It uses Playwright and Selenium to navigate dyn and switch into payment iframes. It also extracts VPAs from QR codes using Pillow and pyzbar gathered 549 raw files, deduplicating them down to 161 unique, validated payment records."*

SLIDE 9: AGENT 2: STREAMING & STORAGE AGENT

What It Is: The event ingestion broker managing streaming delivery to Bronze storage.

How It Works: Scrapers act as Producers publishing JSON to topic 'payment_raw'. A Consumer worker polls the topic and writes JSON to MinIO Bronze.

Why This: High-frequency scrapers can overload database writes. Kafka buffers the incoming message streams asynchronously, ensuring zero lost records.

Why Not Other: RabbitMQ deletes messages once consumed, which prevents historical re-reading. Kafka retains data on disk, enabling stream replays to rebuild the Lakehouse.

Spoken Script:

> *"Next is Agent 2, our Ingestion Broker. It uses Apache Kafka to separate scraping from databas scraper publishes raw payloads to the payment_raw topic, and a consumer saves them to MinIO Br This guarantees zero data loss, even if our database goes offline."*

SLIDE 16: INTERACTIVE ANALYTICS DASHBOARD

What It Is: The Streamlit frontend analyst console exposing risk metrics.

How It Works: Reads Silver Parquet data, renders category donut charts and horizontal bar graphs, and provides a chat assistant interface.

Why This: Streamlit is written in pure Python, integrating natively with Pandas DataFrames and ML plotting libraries.

Why Not Other: React/Angular require separate frontend servers, REST APIs, and JS development, increasing project technical debt.

Spoken Script:

> *"Ma'am, Slide 16 shows our Streamlit Interactive Analytics Dashboard. It provides KPI scorecards, horizontal gateway bar charts, and a risk logging table. Analysts can easily filter all metrics by sele betting sites from the dropdown."*

SLIDE 17: CODEBASE PACKAGE LAYOUT

What It Is: The repository folder hierarchy structure.

How It Works: Divided into: data_collection/ (scrapers), streaming/ (Kafka), backend/ (Spark), ml/ (ML models), rag/ (LLM), and verification/ (auditor).

Why This: Clean modularity. Allows multiple developers to work on separate modules concurrently without git conflicts.

Spoken Script:

> *"Our codebase is structured into decoupled packages. This makes our repository clean and modular. update a scraper rule or retrain a model, we edit only that specific folder without affecting th application."*

SLIDE 18: MEDALLION LAKEHOUSE DATA FLOW

What It Is: The data flow from raw collection in Bronze to cleaned Silver.

How It Works: Bronze JSON -> Spark Cleaning -> Silver Parquet -> ML Risk -> FAISS -> RAG QA.

Why This: Running ML models or RAG vector search directly on messy scraped DOM HTML is impossible. Medallion structures the pipeline progressively.

Spoken Script:

> *"This is our Medallion Lakehouse data flow. Raw JSON files in the Bronze layer are refined by PySpa Parquet format. Our machine learning risk models and FAISS vector database read from this clean ensuring all predictions and RAG answers are factual."*

SLIDE 19: FUNCTIONAL RESULTS SUMMARY

What It Is: The final performance scores and database metrics.

How It Works: Deduplication: 99.8%. Random Forest Accuracy: 84.8%. Flagged Anomalies: 17 total. RAG Grounded Accuracy: 100% verified.

Spoken Script:

> *"This slide summarizes our final project results: a 99.8% data deduplication rate via PySpar classification accuracy via Random Forest, 17 isolated anomalies via Isolation Forest, and su similarity lookups using FAISS."*

SLIDE 20: KEY PLATFORM ADVANTAGES

What It Is: The core architectural benefits of our local-first implementation.

Why This: Local execution via Ollama and Llama 3 keeps sensitive banking data on-premises, with zero external cloud API costs.

Why Not Other: Cloud-based APIs (OpenAI/Claude) charge per token and send sensitive payment endpoints to third-party servers, violating compliance rules.

Spoken Script:

> *"Our main platform advantages are sub-second processing, full audit logs, and zero data loss. By run locally via Ollama, we ensure complete data privacy for sensitive payment details while incurring ze costs."*

SLIDE 21: LIMITATIONS & RISK FACTORS

What It Is: An honest evaluation of the project's current boundaries.

How It Works: Identifies DOM selector fragility, small Bank Transfer class support (15 records), and local LLM hardware requirements (16GB RAM).

Spoken Script:

> *"We identified a few project limitations: web scrapers must be updated if target sites change their pag*
> *direct bank transfer category had a small count of 15 records causing recall variance, and local LL*
> *least 16GB of system RAM."*

SLIDE 22: FUTURE SCOPE & ENHANCEMENTS

What It Is: The roadmap for scaling and securing the platform.

1. Image OCR: Use Tesseract to read text off QR code screenshots, automating canvas data ingestion.

2. Blockchain: Use Web3.py to query blockchain transaction ledgers, tracing crypto money flows.

3. Encryption: Encrypt VPA and Bank columns inside Parquet using AES-256 to secure data.

4. MLOps: Automate model retraining pipelines as new payment configurations arrive.

Spoken Script:

> *"For future work, we plan to implement Computer Vision OCR to read QR codes, Blockchain Ana crypto wallets, AES-256 Column-Level Encryption to secure bank details, and automated model retra records arrive."*

SLIDE 23: PROJECT CONCLUSION

What It Is: The final wrap-up of the academic project.

How It Works: Summarizes the successful integration of scraping, streaming, Spark cleaning, ML modeling, and local vector search.

Spoken Script:

> *"In conclusion, our project successfully integrates web scraping, real-time streaming, PySpark clea modeling, and local RAG. We achieved a 99.8% deduplication rate and 84.8% classification accura private, secure payment intelligence platform can be built entirely on open-source tools."*

SLIDE 24: THANK YOU

What It Is: The final slide inviting Q&A.

Links Provided: GitHub Code Repository and Streamlit Live Application URL.

Spoken Script:

> *"This concludes our presentation. Our code is available on our GitHub repository, and the dashboard on Streamlit Cloud. We thank Ms. Krishnaveni and C-DAC Hyderabad for their guidance. We are questions."*